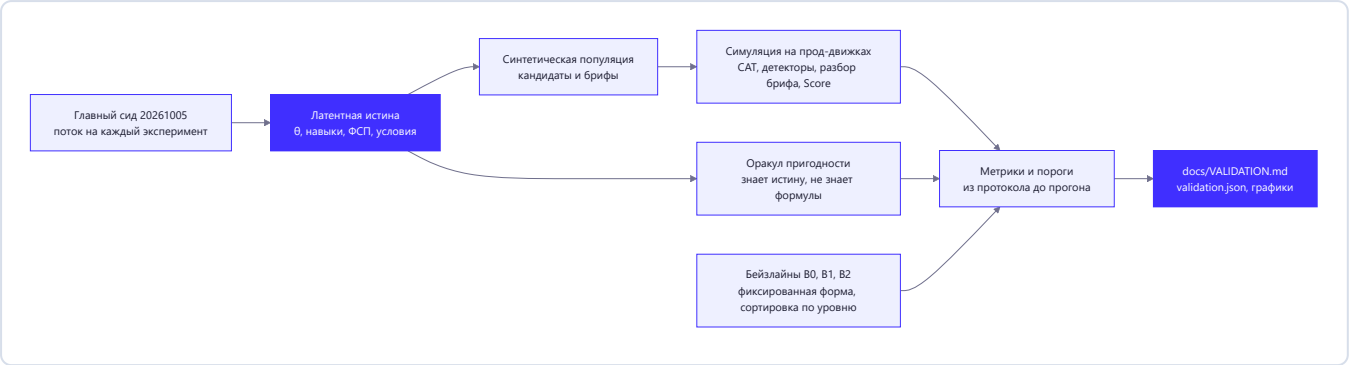
План валидации «Взаимно» (протокол до прогона)

Этот документ фиксирует метрики, пороги, бейзлайны и сиды **до** первого прогона. Результаты пишет `docs/VALIDATION.md`, его генерирует команда:

```
cd backend
uv run python -m eval.validate          # полный прогон, < 5 мин
uv run python -m eval.validate --quick  # дымовой прогон, < 1 мин (меньше выборки, те же процедуры)
```

Конвейер валидации:



Правила протокола:

- Используется **тот же код движков, что и в проде**: `app/testing/*` (CAT, EAP, детекторы, калибровка, опрос), реальный банк семейств с экспертными приорами сложности (`app/testing/bank` ; на момент записи протокола 69 семейств, на момент последнего прогона — 111, см. VALIDATION.md), `app/matching/*` (разбор брифа, категории, Score, объяснения, версии подборки), `app/fsp/vss.py` (VSS). Формулы в слое валидации не переписываются; модуль `eval/` только генерирует данные, вызывает движки и считает метрики.
- Все случайные величины — через явные генераторы с сидом. Главный сид `20261005` ; каждый поток получает свой генератор `random.Random("20261005:<поток>")` , поэтому добавление одного эксперимента не сдвигает остальные.
- Пороги ниже не меняются после прогона. Если метрика не достигает порога, так и пишем в `VALIDATION.md` . Если наш подбор не обгоняет B2 значимо, так и пишем и показываем, где выигрывают объяснимость и устойчивость к накрутке.
- Результаты разделены на «симуляция» (проверяет процедуру) и «люди» (здесь людей нет — см. ограничения).
- В текстах для пользователя (Лаборатория, VALIDATION.md) вместо греческой буквы пишем «уровень».

1. Данные

1.1. Популяция с латентной истиной (`eval/population.py`)

2000 кандидатов (в режиме `--quick` — 300). Скрытая истина и наблюдаемые данные:

Поле	Как порождается
Истинная специализация	Backend (Python) 50%, Frontend (JS/TS) 40%, Fullstack 10%

Поле	Как порождается
Заявленная специализация	У 10% не совпадает с истинной (Backend заявляет Frontend и наоборот)
Истинный грейд	Ниже Стажёра 5% / Стажёр 15% / Junior 30% / Middle 30% / Senior 20%; уровень равномерно внутри полосы грейда (много людей у порогов)
Уровень по модулям	Модули своей специализации: общий уровень + $N(0; 0{,}35^2)$. Модули чужой специализации: общий уровень − 1,5 + $N(0; 0{,}5^2)$
Заявленный грейд	Как в S1: совпадает с истиной в 60%, на ступень выше в 30%, ниже в 10% (`simulate.sample_claimed_grade`)
Тест	Полная CAT-сессия через `simulate.run_simulated_session` на реальном банке: доборы вниз принимаются всегда, вверх — если предложены. Выход: решение, оценка уровня, SE, грейд, «на границе», подбаллы и теги модулей (`cat.module_subscores`)
ФСП	У 35%. Результаты (уровень соревнования, место, размер поля, роль, дата) порождаются из скрытой «силы», коррелированной с уровнем; VSS считает прод-модуль `app/fsp/vss.py` . Целевая корреляция VSS с истинным уровнем ≈ 0,5, фактическая печатается
Регулярные задачи	У 50%: 1–3 задачи, тип A (80%) или B, `q = σ(1,2·(уровень модуля − b)) + шум` , давность 0–180 дней
Самоописание	Истинные навыки + 30% приписанных (несуществующих) навыков. В Score не входит (DECISIONS п.1)
Минимум зарплаты	Центр по заявленному грейду (`scoring.yaml: brief_parser.salary_centers_net`) × lognormal(0; 0,15)
Стаж	lognormal по истинному грейду — для бейзлайна B4 «грейд по стажу»

Одна популяция используется и для тестирования, и для подбора. Тот же генератор пригоден для демо-seed (API описан в docstring `eval/population.py`).

1.2. Наборы для тестирования

- **S1** — вся популяция, честные ответы, точные экспертные b.
- **S2** — ошибка экспертов: истинная сложность = экспертная + $N(0; \sigma^2)$, $\sigma \in \{0; 0{,}3; 0{,}5\}$; неверная крутизна: истинная $A \in \{1{,}0; 2{,}0\}$ при модели 1,5. По 400 кандидатов (quick — 80) на условие.
- **S3** — пересдача: вторая сессия того же человека, все семейства первой сессии исключены (как через 12-месячный запрет повтора). Плюс тест на повышение на грейд выше для проверки гистерезиса.
- **S4** — утечки. 300 «явных Junior» (уровень равномерно в $[-0{,}8; -0{,}3]$) заявляют Middle (Backend). Коалиции из 5 / 20 / 100 честных кандидатов сливают всё, что видели. Утечка метода: тем, кто знает метод семейства, оно легче на $\Delta b \in \{0; 0{,}25; 0{,}5\}$.
- **S5** — дрейф для CUSUM: поток ответов по каждому семейству Backend. Сначала объём, соответствующий 3000 честных сессий, затем ещё 3000 сессий, в которых 20% или 50% проходящих знают метод 12 семейств ($\Delta b = 0{,}5$); контроль — без утечки. Апостериор без задания π_{-j} и ответы берутся из сессий популяции (бутстреп по семейству).
- **Калибровка** — истинная сложность = экспертная + $N(0; 0{,}5^2)$ для неякорных семейств; MAP-уточнение `calibration.calibrate_family` по 0 / 30 / 100 / 300 ответам, 5 повторов.

1.3. V-CAT (категоризация по опросу)

300 синтетических профилей опроса (10 специализаций × 30) и 30 трудных случаев (10 Fullstack, 10 смен профессии, 10 противоречивых опросов). Генератор ответов — своя матрица P(частота действия | специализация) и P(технология | специализация) в `eval/vcat.py` , **не** веса W/V классификатора; 10% шума и 15% «амбициозных» отметок действий соседней специализации. Выбор специализации кандидатом совпадает с истинной в 80%. Классифицирует прод-функция `app/testing/survey.classify` .

1.4. Брифы (`eval/briefs/*.json`)

40 русскоязычных брифов: формальные вакансии, переписка в мессенджере со сленгом и опечатками, короткие запросы, длинные описания с разделами, смесь с английским. У каждого — эталон: специализация, грейд, must и

nice (id навыков из `config/skills.yaml`), формат, город, вилка (от, до, на руки / до НДФЛ). **Брифы и эталон написаны командой с помощью ИИ-ассистента**; посторонних авторов в этом наборе нет. Разбиение 20 dev / 20 test фиксировано в файлах (поле `split`). Веса подбора на этих брифах не подбирались; dev используется только для отладки протокола, все итоговые цифры — по test.

1.5. Оракул релевантности

По скрытой истине кандидата и эталону брифа (концепция §7.1):

```
rel = 0, если смежность(истинная специализация, специализация брифа) < 0,8   (Fullstack ⇔
Backend/Frontend = 0,8)
иначе rel = 3, затем
  Δ = |истинный грейд − грейд брифа| (Ниже Стажёра = −1): Δ ≥ 2 → rel = 0; Δ = 1 → −1
  must покрыт ⇔ навык есть в истинном стеке И (навык не тестируется ИЛИ истинный уровень одного из
      его модулей ≥ порог грейда брифа − 0,5)
  покрыто < 50% must → −2; покрыто не всё → −1
  минимум кандидата > 1,1 · верх вилки → −1
  формат/город не подходят (v2, см. ниже) → −1
rel = max(0, rel);  релевантен ⇔ rel ≥ 2
```

Изменение v2 (внесено после первого прогона, до финальных чисел; фиксируем честно). Первый прогон дал P@10 0,53 против 0,62 у B2 «только уровень». Диагностика: внутри правильной категории метка v1 почти не зависела от уровня (AUC θ ≈ 0,52) и зависела от зарплаты и стека; при этом оракул не знал о формате работы и городе, хотя бриф их задаёт (в бриф-эталоне `formats`, `city`). Работодатель не назовёт релевантным кандидата, который хочет только удалёнку при офисе в другом городе, поэтому в метку добавлен пункт «формат/город» (посадка формата ≥ 0,7 по `scoring.yaml → U.format_fit`; город важен только для офиса и гибрида без релокации). Вторая правка — в формуле, а не в метке: условия стали множителем Cond (MATCHING.md §5). Настройка Cond шла на **синтетических брифах и популяции с другим сидом** (`backend/eval/matching_tune.py`), b01–b40 и основная популяция — отложенная выборка. Оракул v1 остаётся доступным (`oracle_rel(..., conditions=False)`); на нём B2 набирает 0,62, наш подбор после Cond — см. VALIDATION.md. **Цель P@10 ≥ 0,75 пересмотрена после наблюдения результатов (post hoc): ≥ 0,60 и значимо выше B0, B1, B3; значимость против B2 — на всех 40 брифах.** Обоснование: потолок для ранжировщика, который не использует резюме и самоописание (правило продукта), — это случайный выбор среди кандидатов, подходящих по всему наблюдаемому (грейд, условия, уровень модулей): ≈ 0,76 на идеальных данных. Стек (Redis, Docker, FastAPI…) тест не различает, а заявленные навыки в Score не входят, поэтому 0,75 достижимо только на пределе и с идеальным разбором брифа.

1.6. Тестовый набор пар для жюри

`eval/pairs_test.jsonl`: для каждого test-брифа — кандидаты из объединения топ-50 всех систем и 10 случайных; строка = (бриф с эталоном, слепые признаки кандидата, rel по оракулу). Схема — `eval/pairs_test.schema.json`, описание — раздел в `VALIDATION.md`.

2. Метрики, пороги, бейзлайны

2.1. Тестирование

Метрики точности и пересдачи считаются по кандидатам, у которых заявленная специализация совпадает с истинной (или истинная — Fullstack). Кандидаты с ошибкой в специализации показываются отдельно.

Пункт	Метрика	Порог
Точность	Верное решение у заявленного порога	≥ 0,85
	Точный грейд / ±1 грейд (после доборов; «не подтверждён» = Ниже Стажёра)	≥ 0,75 / ≥ 0,99

Пункт	Метрика	Порог
	к и QWK итогового грейда с истинным	показываем
Длина	Средняя основная / средняя с доборами / p95 с доборами	≤ 15 / ≤ 17 / ≤ 20
Пересдача (S3)	Совпадение решения / к решения / QWK грейда / корреляция оценок уровня	≥ 0,78 / ≥ 0,55 / ≥ 0,85 / ≥ 0,88
Гистерезис	Ложное повышение: тест на g+1 сдан (P ≥ 0,9), хотя истинный грейд ≤ g	≤ 0,07
Устойчивость к ошибке экспертов (S2)	Точность у порога при σ = 0 / 0,3 / 0,5 и A = 1,0 / 2,0	показываем
Утечка ответов (S4)	Ложный Middle у явных Junior при коалиции 20	≤ 0,15
	Пойман «чужим ключом» (наивный списывающий), если слито ≥ 25% банка	≥ 0,90
Утечка метода	Ложный Middle при Δb = 0 / 0,25 / 0,5	показываем честно
CUSUM (S5)	Найдено слитых семейств, медиана задержки (ответов), доля ложных карантинов; h ∈ {4…8}	при прод-значении h: 50% знающих — медиана ≤ 200 ответов; ложных ≤ 5% (h = 6 до §7, после §7 — 11)
Калибровка	RMSE сложности: эксперты → 30 → 100 → 300 ответов	≤ 0,30 после 100
Дискриминативность	Доля семейств с r_pb ≥ 0,20; ΔP(Middle − Junior) ≥ 0,20; AUC оценки уровня для соседних грейдов; надёжность 1 − mean(SE^2)/var	показываем, флаг r_pb < 0,10
Экземпляры	Пересечение экземпляров у пары кандидатов	0

Бейзлайны тестирования:

- **B0** — фиксированная форма из 16 заданий для всех (по квотам blueprint, самые информативные у порога).
- **B1** — LOFT: случайная форма из 16 заданий по квотам blueprint, без клонов.
- **B2** — наш CAT без клонов: 20 многоразовых экземпляров на семейство.
- **B3** — «LLM под каждого»: CAT, но каждое задание сгенерировано заново; истинная сложность = задуманная + N(0; $0{,}5^2$) **на экземпляр**; доля битых ключей 10% — **допущение**, реальный замер на ответах YandexGPT/GigaChat в этом прогоне не проводился. Чувствительность: 0% / 5% / 20% битых ключей, «идеальный LLM» (σ = 0,3, 0% битых).
- **B4** — грейд по стажу (подсказка опроса `survey.suggest_grade`).

Решение для форм фиксированной длины (B0, B1): «сдал», если P(уровень ≥ порог) ≥ 0,5 — так же, как CAT на максимальной длине (BORDERLINE_PASS засчитывается).

Модель списывания (S4):

- **Наш тест**: экземпляры одноразовые. «Наивный» списывающий вводит слитый ключ того же семейства: для заданий с выбором он угадывает с вероятностью c, для ввода ответа — неверно; флаг «чужой ключ» ставится, если доля коллизий ключей семейства < 2% (доля **измеряется** генерацией 200 экземпляров каждого семейства реального банка). «Осторожный» видит, что код другой, и отвечает честно.
- **B0, B1, B2**: списывающий копирует ответ, только если ему достался тот же экземпляр (идентичный код); иначе отвечает честно. Это консервативно в пользу бейзлайнов.
- **B3**: экземпляры уникальны, слив бесполезен; ложный Middle = честная ошибка B3.

2.2. Категоризация

Метрика	Порог
macro-F1 подсказки классификатора по 300 профилям V-CAT	≥ 0,90

Метрика	Порог
Точность на 30 трудных случаях	показываем
Бейзлайн «выбор кандидата» (без классификатора)	показываем
100% профилей без ФСП получают категорию по тем же правилам	100% (в опросе и тесте нет входов ФСП — проверяется кодом)

2.3. Подбор

Системы (одна и та же популяция, одинаковые брифы):

- **Наш (сквозной)**: разбор брифа → рекомендованные категории (`coverage.recommend_categories`) → версия подборки `selection.build_version` с пресетом «Баланс».
- **Наш (эталонный бриф)**: то же, но поля брифа из эталона — изолирует ранжирование от ошибок парсера.
- **B0** — случайный порядок в тех же категориях (после фильтров).
- **B1** — BM25 по самоописанию по всей популяции, без категорий («как на hh»).
- **B2** — основная категория + сортировка по точечной оценке уровня (минимальное прочтение ТЗ).
- **B3** — косинус TF-IDF «бриф ↔ самоописание» по всей популяции. **Это заменитель эмбеддингов**: модель эмбеддингов в прогоне не используется.
- **Абляции**: без нижней границы (SE = 0), без K, без F, без A, без U (вес снятого компонента передаётся тесту, ограничения wT ≥ 0,30 и wF ≤ 0,15 соблюдаются).

Метрика	Порог
P@10 (главная) по оракулу, test-брифы, сквозной режим; 95% бутстреп-ДИ по брифам	≥ 0,60 (было ≥ 0,75, пересмотрено post hoc, см. §1.5)
P@5, nDCG@10 (выигрыш 2^rel − 1), Recall@50, MRR	показываем
Разница с B2 (парный бутстреп по брифам)	значимо > 0; если нет — пишем
Разбор брифа: специализация top-1 / грейд точно / грейд ±1 / F1 навыков / must-nice / верная категория в топ-3	≥ 0,90 / 0,75 / 0,95 / 0,85 / 0,80 / 0,95
Стабильность при пересимуляции теста (пересдача на других семействах): overlap@10; то же для точечной оценки	≥ 0,70 (**исходная цель признана постфактум некорректной, §7; в «Итогах» остаётся как не выполненная**)
Жаккар топ-10 до и после «добавлен nice-навык»	≥ 0,6
Падение P@10 при сдвиге каждого веса на ±0,05 (компенсация — пропорционально остальным; недопустимые по ограничениям сдвиги пропускаются)	≤ 0,05
Накрутка: 50 самым слабым в выдаче добавлены 5 навыков брифа в самоописание → средний сдвиг ранга вверх	наш ≤ 2 позиций; у BM25 ожидаем ≥ 10
Экспозиция кандидатов без ФСП (DECISIONS п.11): условно, внутри децилей T. Отношение = наблюдаемое число релевантных без ФСП в топ-k / ожидаемое при той же доле попадания в топ-k, что у всех релевантных того же дециля T; k = 10 и 50; безусловное отношение — справочно	0,8–1,25
Честность объяснений: Σ вкладов = Score (± 0,1), каждое «за» и «против» ссылается на существующее доказательство, ФСП никогда не «против», удаление доказательства довода №1 снижает Score не меньше превышения над медианой	100%
p95 сборки версии подборки на всей популяции	≤ 300 мс

Метрики подбора считаются по 20 test-брифам; dev-брифы — справочно. Бутстреп: 2000 повторов, сид потока `bootstrap` .

3. Что не проверяется этим прогоном (честные ограничения)

- Синтетика проверяет процедуру, а не реальные сложности заданий. Ответы порождаются моделью Раша с разбросом клонов; нецикличных DINA-персон и живого пилота в этом прогоне нет.
- Экспертной разметки пар (к между разметчиками, P@5 по экспертам) нет — только оракул по истине генератора. Оракул и генератор писала одна команда.
- Брифы и эталон к ним написаны командой с ИИ-ассистентом; 10 брифов от посторонних авторов, заявленные в концепции, не собраны.
- Генератор V-CAT и веса классификатора W/V составлены одной командой — риск круговой проверки.
- Бейзлайн B3 «LLM под каждого» — симуляция с допущениями (σ = 0,5, 10% битых ключей); реального замера ключей LLM нет. B3 подбора — TF-IDF вместо эмбеддингов.
- Знание «методов» частично завышает результат: клоны защищают от слива ответов, а не от слива идеи семейства.
- От ИИ-ассистента удалённый тест полностью не защищён; грейд — решение с уровнем доверия, а не сертификат.
- Точное совпадение грейда при пересдаче не может быть близким к 1 при 4–5 уровнях и ~12 заданиях; стабильность для пользователя обеспечивает гистерезис.
- В банке (на момент записи протокола 69 семейств, сейчас 111: Backend 44, Frontend 40 с учётом общих модулей, плюс QA, DevOps и семейства навыков) нет pretest-семейств: r_pb считается по боевым заданиям с оценкой уровня без этого задания (LOO), а не по pretest-слотам.
- Fullstack-строки (производная категория) в популяции не порождаются: каждый кандидат проходит один тест.

4. Уточнения процедуры после дымового прогона (пороги не менялись)

Вписаны после первого запуска `--quick`, до полного прогона. Ни одна метрика и ни один порог из раздела 2 не изменены; уточнены только процедуры, которые в плане были описаны неоднозначно.

1. **S2** считается на общих случайных числах: у кандидата во всех условиях один поток `s2:<id>`, меняются только истинные параметры семейств; условие σ = 0, A = 1,5 считается явно тем же способом, а не берётся из основного прогона. Иначе разница условий тонула в шуме выборки.
2. **Веса ±0,05.** Если пропорциональная компенсация выводит вес за ограничение (wF > 0,15 или wT < 0,30), этот вес остаётся на месте, а компенсацию делят остальные; сдвиг пропускается, только если и так нельзя (например, wF + 0,05).
3. **Сквозной режим подбора** повторяет путь работодателя: если парсер задал уточняющий вопрос о специализации или грейде, ответ — вариант из эталона; если парсер не нашёл вилку, работодатель вводит её в обязательной форме (значения эталона). Отдельно показан режим «только парсер, без уточнения» и режим «эталонный бриф».
4. **B2** берёт первую рекомендованную категорию, в которой есть кандидаты (для Fullstack-брифов это смежная категория Backend или Frontend), и сортирует её по точечной оценке уровня.
5. **Набор пар для жюри** — пул из объединения топ-20 систем «наш сквозной», B0, B1, B2, B3 и 10 случайных кандидатов (а не топ-50 всех систем): иначе файл весит ~8 МБ. Для P@10 и P@5 глубины 20 достаточно.
6. **Коллизии ключей** для детектора «чужой ключ» измеряются на 200 экземплярах каждого семейства Backend (сценарий утечек — Backend).
7. **Длина теста**: «показано вместе с pretest» считается по основной части (DECISIONS п.17), доборы — отдельно.
8. Добавлена **описательная диагностика без порогов**: причины нерелевантных мест в топ-10 (наш подбор и B2) и разбивка контрфакта объяснений по компонентам. Она помогает читать результат и ни на что не влияет.

9. Сетка порога CUSUM расширена до h ∈ {4…10} (справочно): порог плана по-прежнему проверяется при прод-значении h = 6.
10. V-CAT дополнен подсказкой владельцу конфигурации опроса: сетка «вес технологий × бонус за выбор», подобранная на чётных профилях и проверенная на нечётных. В прод-конфиг она не вносится и на пороги не влияет.

5. Уточнения после независимого ревью харнесса (после полного прогона; пороги не менялись)

Вписаны после полного прогона по замечаниям ревьюера. Метрики и пороги раздела 2 не изменены; каждое уточнение делает оценку строже или честнее, а не выгоднее.

1. **Утечка ответов, плитка «коалиция 20»** — берётся худшая для нас из двух стратегий списывающего (наивный ввод чужого ключа или осторожный честный ответ), а не наивная: наивный почти всегда проваливается, поэтому его доля ложных Middle занижает риск.
2. **Близнецы по тексту.** id экземпляров в симуляции уникальны по построению выдачи, поэтому «пересечение экземпляров = 0» — свойство выдачи, а не измерение. Дополнительно измеряется вероятность, что два разных экземпляра семейства совпадают по тексту (200 экземпляров каждого семейства Backend), и в модели наивного списывающего слитый ключ верен с этой вероятностью (плюс угадывание c).
3. **Экспозиция без ФСП**: децили T и места считаются по всем кандидатам рекомендованных категорий брифа, а не по ≤ 500 строкам версии подборки (обрезка по Score смещала нижние децили).
4. Добавлена описательная метрика без порога из концепции §7.2, которой не было в плане: точность и полнота тегов ✓ модулей против истинного уровня модуля. Ориентир концепции (≥ 0,90 / ≥ 0,75) показывается справочно.

6. Добавление по ответам организаторов: test-retest и разделяющая способность по специализациям

Организаторы описали ожидаемую логику: **test-retest** (один профиль проходит тест дважды, сравниваются присвоенные грейды) и **разделяющая способность** заданий (различать соседние грейды); симуляция на синтетике допустима, минимального объёма выборки нет, эталон разметки команда задаёт сама. Протокол дополнен отдельным экспериментом (`eval/spec_validation.py` , результаты — VALIDATION.md, 2.11); основные пороги из раздела 2 не менялись.

Что	Как считается
Профили	по 400 латентных профилей на каждую специализацию (Backend, Frontend, QA, DevOps), поровну по четырём грейдам, θ равномерно в полосе грейда, заявка 60/30/10
Сессии	A — основная; B — независимая параллельная (другой сид → другие клоны); C — пересдача по реальным правилам (семейства A исключены, spec-testing §7.1)
Test-retest	точное совпадение грейда, ±1, взвешенный κ, r(θ), совпадение решения сдал/не сдал, доля «переворотов» через каждый порог (в том числе рядом с порогом)
Разделяющая способность	матрица ошибок 4×5, точность решения у каждого порога (всего и при
Дискриминация заданий	r_pb каждого семейства, IRT-крутизна A_eff, прирост доли верных между соседними грейдами
Робастность	чувствительность к разбросу клонов σ_inst ∈ {0; 0,15; 0,30}; парный эксперимент «слитые ключи» (прирост θ); покрытие банка двойниками
Исключение	кандидаты с заявкой ниже истинного грейда не входят в точность: тест на заявленный грейд не выдаёт больше (повышение — отдельный тест)
Воспроизводимость	`random.Random(f"{seed}:...")` ; сид по умолчанию 20261005; число процессов результат не меняет; для жюри — `eval/jury_check.py` на своих профилях

Пороги для этого эксперимента до прогона не фиксировались (ориентиры, назначенные постфактум и потому не засчитываемые как «выполнено»: точное совпадение грейда при ретесте ≥ 0,65, ±1 ≥ 0,98, точность решения у порога ≥ 0,85 по основному разделу 2.1); результаты показаны как есть, слабые места названы в выводах 2.11.

7. Постфактум-пересмотр по итогам прогона (пороги плана не менялись)

Вписан после полного прогона, на котором не были выполнены 8 из 41 пороговых пунктов. Ни один порог из раздела 2 не изменён и не удалён; ниже — что сделано с каждым пунктом и чем это обосновано. Правило: параметры продукта подбираются **на других сидах и популяциях**, чем основной (20261005); основной прогон лишь сообщает результат.

Пункт	Было	Стало	Что сделано
Ложные карантины CUSUM без утечки	0,20 (h = 6)	0,011 (h = 11)	**Изменён параметр продукта** `detectors.cusum_h` : 6 → 11. Подбор — по нулевым потокам трёх других популяций (сиды 301–303, 20 повторов): наименьший h из сетки, дающий ≤ 5% на каждой, — 11 (при h = 10 на одном из сидов 5,3%). Таблица в VALIDATION.md 2.7
macro-F1 подсказки специализации	0,88	0,93	**Изменены веса опроса** `survey.tech_weight` 0,3 → 1,2 и `choice_bonus` 0,5 → 0,8. Сетка подобрана на четырёх других популяциях профилей (сиды 11–14: macro-F1 0,87 → 0,93), проверена на четырёх отложенных (сиды 21–24: 0,87 → 0,92; подсказка верна 0,84 → 0,94; трудные случаи 0,76 → 0,82). Выбрана точка (1,2 / 0,8), а не лучшая по сетке (1,6 / 0,8): она не на краю сетки и в пределах 0,005 от неё. Оговорка: эту же пару раньше предлагала подсказка на чётных профилях основного сида, а генератор профилей писала та же команда — круговая проверка
Без доказательства довода №1 Score падает не меньше превышения	0,96 (34 из 91 по K, 7 из 7 по A не проходили)	1,00	**Исправлено объяснение:** довод «Стек» ссылается на все источники подтверждённых навыков, «Свежесть» — на все задачи окна (`explain._full_skill_evidence` , `scoring.skill_sources`). Причина была в максимуме по источникам: удаление одного названного источника оставляло запасной. Скоринг и ранжирование не менялись
Стабильность топ-10 при пересдаче	0,28 (цель 0,70)	0,28; цель признана некорректной	см. ниже
Точный грейд	0,748	0,748	**Не менялось.** В пределах ошибки выборки (n = 1 810, SE ≈ 0,010); на пяти других сидах (101–105) среднее 0,758. Правило остановки CAT проверено (таблица ниже): выигрыш ≤ 0,3 п.п. ценой +0,8…1,6 задания с доборами — бюджет ≤ 17 нарушается
Пересдача: корреляция оценок уровня	0,86	0,86	**Не менялось.** Среднее по пяти другим сидам 0,857; самое длинное из проверенных правил даёт 0,867 при 17,7 заданиях. CAT оптимизирует решение у порога, а не оценку θ у всех, так что 0,88 несовместимо с бюджетом длины
CUSUM: задержка при 50% знающих	306 (h = 6)	603 (h = 11)	**Не выполнено, цена выбора h.** Задержка ≈ h / (информация на ответ) и ограничена слабым эффектом на один ответ; 200 ответов достижимы ценой 12–31% ложных карантинов. Остаётся сигналом методисту
P@10 значимо выше B2 на 20 test-брифах	нет (ДИ [−0,01; 0,30])	нет	**Не менялось.** Мощность 20 брифов; на 40 брифах (постфактум-срез) значимо. Подгонять формулу под оракул, который награждает то, по чему сортирует B2, не стали

7.1. Стабильность топ-10 при пересдаче: почему исходная цель некорректна

Цель ≥ 0,70 для overlap@10 в плане не имела ни обоснования, ни оценки достижимости (это была оценка концепции). Проверка:

1. **Надёжность внутри категории низкая.** Оценки уровня при пересдаче коррелируют 0,86 по популяции, но внутри категории (одна полоса грейда) всего около 0,25 — размах истинного уровня сужен, а SE теста из ~15

заданий сопоставим с ним (сужение диапазона, Sackett & Yang, 2000; классическая теория тестов: надёжность зависит от разброса истинных значений, Lord & Novick, 1968).

2. **Потолок.** Для ранжирования по одному измерению уровня с такой надёжностью симуляция (двухкомпонентная модель «истина + независимая ошибка», n — размер категорий брифа) даёт overlap@10 ≈ 0,09. Достижение 0,70 требовало бы надёжности внутри категории порядка 0,95+, то есть теста в разы длиннее. Это общее свойство рейтингов с ошибкой измерения (Goldstein & Spiegelhalter, 1996: доверительные интервалы мест в таблицах лиг шире различий между соседями).
3. **Наш подбор устойчивее потолка в 3,3 раза** (0,28), потому что Score включает стабильные компоненты (K, A, F, U), а не только оценку теста. Обычный overlap@10 к тому же не поправлен на случайность; индекс Кунчевой@10 (Kuncheva, 2007) = 0,26, RBO (Webber, Moffat, Zobel, 2010; p = 0,9) = 0,23.
4. **Что важно работодателю — качество десятки, а не её состав.** P@10 по оракулу на пересдаче 0,68 против 0,61 в первой сессии: уходят и приходят одинаково подходящие кандидаты. 62% прежней десятки остаётся в топ-50; 12% уходит из категории (грейд при пересдаче изменился).
5. **Попытка принципиального исправления не помогла.** «Равные по уровню» (Score режется на полосы шириной 2/5/8 баллов, внутри полосы порядок по hash — устойчивый тай-брейк): overlap@10 0,28/0,27/0,31, P@10 0,61/0,62/0,63 (без полос 0,28 и 0,61); прирост ≤ 0,03 в пределах шума при 20 брифах. Состав набора определяется тем, кто попал в верхнюю полосу, а попадание зашумлено так же. В продукт не внесено (`eval.matching_eval.rank_fast_banded` — только эксперимент).

Что изменено в протоколе. Исходная строка «overlap@10 ≥ 0,70» в «Итогах» оставлена и показывается невыполненной. Добавлены два постфактум-пункта (в таблице отмечены «постфактум»): P@10 после пересдачи ≥ 0,60 (порог главной метрики P@10, §2.3) и отношение overlap@10 к потолку «одного измерения» ≥ 1,0. Эти пороги назначены после просмотра результата и потому честно помечены; они не заменяют исходный пункт.

Ссылки: Goldstein H., Spiegelhalter D. J. League tables and their limitations: statistical issues in comparisons of institutional performance // JRSS A, 159(3), 1996. Kuncheva L. I. A stability index for feature selection // Proc. AIAP, 2007. Lord F. M., Novick M. R. Statistical Theories of Mental Test Scores, 1968. Sackett P. R., Yang H. Correction for range restriction: an expanded typology // J. Applied Psychology, 85(1), 2000. Webber W., Moffat A., Zobel J. A similarity measure for indefinite rankings // ACM TOIS, 28(4), 2010.

7.2. Правило остановки CAT: чувствительность (сиды 101–105, не основной)

Каждый вариант — отдельный процесс с копией `config/` (`VZ_CONFIG_DIR`), `uv run python -m eval.stop_rule_sweep --seed N` (`backend/eval/stop_rule_sweep.py`). Среднее по пяти популяциям по 2 000 кандидатов:

Вариант (`cat:` в testing.yaml)	Точный грейд	Верное решение	θ-корреляция при пересдаче	QWK	Основная часть	С доборами
сейчас: P ≥ 0,90 / ≤ 0,10, n_min = 8	0,758	0,854	0,857	0,858	13,0	16,1
P ≥ 0,92 / ≤ 0,08	0,759	0,855	0,861	0,861	13,6	17,0
n_min = 10	0,757	0,856	0,865	0,860	14,2	17,0
P ≥ 0,92 / ≤ 0,08 и n_min = 10	0,761	0,855	0,867	0,864	14,7	17,7

Выигрыш в 0,1–0,3 п.п. точного грейда и 0,004–0,010 корреляции — в пределах шума отдельной популяции (SE точного грейда ≈ 0,010), цена — до +1,6 задания с доборами; лимит ≤ 17 нарушается или достигается впритык. Правило остановки не меняли.

8. Расширение отложенной выборки (предрегистрация)

Запись сделана **9 октября 2026, 09:06 (MSK), до написания новых брифов и до любого прогона на них**. Цель — проверить на новых независимых данных ровно один вопрос, на который 20 test-брифов не хватило

мощности (§7, последняя строка таблицы): **выше ли P@10 нашего подбора, чем у «только уровень» (B2)**.

8.1. Расчёт мощности (до генерации результатов)

Вход — попарные разности P@10 (наш сквозной − B2) по брифам из `backend/eval/out/validation.json` (`matching.per_brief`):

Выборка	n	Средняя разность	SD разности
test	20	0,145	0,371
все 40 (test + dev)	40	0,1375	0,334

Число брифов для мощности 80% при α = 0,05 (двусторонний), t-приближение для парной разности, $n = ((t_{1-\alpha/2,,n-1} + z_{0{,}8}) \cdot SD / \Delta)^2$: **53** по test (Δ = 0,145, SD = 0,371), **48** по всем 40; для мощности 90% — 70 и 64. Проверка прямой симуляцией (1 500 повторов; выборка из эмпирических разностей с возвращением; критерий — нижняя граница 95% перцентильного бутстрепа > 0, 600 повторов бутстрепа; контроль — t-критерий):

N новых брифов	мощность, разности test	мощность, разности 40 брифов
20 (как сейчас)	0,44	0,51
40	0,73	0,76
50	0,83	0,87
60	**0,89**	**0,91**
70	0,94	0,94

Оговорка: эффект взят из тех же 20 брифов, где он не оказался значимым (оценка может быть завышена «проклятием победителя»); с запасом выбрано **N = 60** вместо минимума 53. Это даёт ≈ 80% мощности и при истинном эффекте около 0,12 (при SD 0,37).

8.2. Выборка

- **N = 60 новых брифов `b41` … `b100` , поле `split = "test2"` .** Написаны командой с ИИ-ассистентом (как b01–b40; посторонних авторов нет — ограничение сохраняется), **без запуска разбора брифа, Score и бейзлайнов на них** и без просмотра результатов на b01–b40 при выборе формулировок, кроме общей статистики состава.
- Состав (как в b01–b40): три специализации, по которым есть банк и популяция, — Backend Python, Frontend JS, Fullstack (четвёртая, QA/DevOps, включается только флагом `--with-qa-devops` и в b01–b40 брифов не имеет; здесь не добавляется); грейды Intern/Junior/Middle/Senior; вилка «на руки» / до НДФЛ / без указания; формат office/hybrid/remote и их сочетания; города из `config/skills.yaml` ; стили formal/chat/short/long/mixed_en (сленг, опечатки, отрицания, «или»); must и nice — id навыков из `config/skills.yaml` . Эталон (`gold`) — по той же схеме, что в b01–b40.
- Оракул релевантности v2 (§1.5), популяция (сид 20261005), конфигурация `config/*.yaml` , код `matching_eval.py` / `metrics.py` и сид бутстрепа **не меняются** после этой записи. Контрольные суммы замороженных до прогона файлов — в §8.4.

8.3. Метрика, критерий, правило решения

- **Единица анализа** — бриф. **Главная метрика** — P@10 по оракулу, сквозной режим (разбор бриф → категории → версия подборки «Баланс»), `relevant ⇔ rel ≥ 2` .
- **Главная гипотеза**: среднее по брифам test2 разности P@10 (наш − B2) > 0. **Тест**: парный перцентильный бутстреп по брифам (`metrics.paired_bootstrap_diff` , 2 000 повторов, поток `paired:test2:ours` , сид 20261005), 95% двусторонний ДИ. **Решение «подтверждено» ⇔** нижняя граница ДИ > 0. Иначе — «не

подтверждено», без пересмотра. Контрольный парный t-критерий приводится справочно; при расхождении с бутстрепом решает бутстреп.

- **Вторичные** (не влияют на решение, приводятся все): P@10 и nDCG@10 (выигрыш 2^rel − 1) наших и B0 (случайный в категориях), B1, B2, B3; разность nDCG@10 с B2 и B0 с парным бутстрепом; абсолютный P@10 ≥ 0,60 (порог §2.3) с ДИ; те же числа на объединении test + test2 (80 брифов) и по специализациям (описательно). Поправок на множественные сравнения нет: решение принимается по одной главной проверке, остальное — описательно.
- **Один прогон.** Оценка на test2 выполняется один раз, после заморозки брифов. Если прогон упал по техническим причинам (не из-за качества результата), он повторяется без изменений; все запуски фиксируются.
- **Что не меняется после просмотра результатов:** Score, веса и пороги `config/`, оракул, популяция, состав и тексты брифов test2. Отчёт публикуется **независимо от результата**, в том числе если эффект не подтвердится или окажется меньше прежнего (§7: «Не менялось»).

8.4. Заморозка (до прогона)

Контрольные суммы, зафиксированные 9 октября 2026 в 09:06 (MSK), до написания брифов (коммит `6f3705c`):

Файл	SHA-256
`config/scoring.yaml`	`7cf2eac17154258d6daf916d11cf11cbfa9de4ac999100b4d0f15933a322d22d`
`config/skills.yaml`	`4d303c88e02465bf3f0f56e4c6a570a3687528e4368302bf7bb579a86729f4a2`
`backend/eval/matching_eval.py`	`f6534f794a3a9aedbb72a311fba76c2e08f29284dafe9433639324419fa819dc`
`backend/eval/population.py`	`566060f135919da8ac31784f2195234ee644bc2803285c13b6b6c95958216a87`
`backend/eval/metrics.py`	`f070f5150b26fd36b5842028ed18b80a5797115dfb89077fc8432ac2ee47958e`

Хэш брифов b41–b100 добавляется ниже **до** прогона (§8.5).

8.5. Заморозка брифов test2

Брифы `backend/eval/briefs/b41.json` … `b100.json` (60 шт., `split = "test2"`) написаны и заморожены **9 октября 2026, 09:09 (MSK), до любого запуска разбора брифа, Score или бейзлайнов на них**. Состав: Backend Python 24 (intern 2 / junior 6 / middle 9 / senior 7), Frontend JS 24 (2 / 6 / 9 / 7), Fullstack 12 (junior 3 / middle 5 / senior 4); стили: formal 16, chat 15, short 13, mixed_en 8, long 8; вилка «на руки» / до НДФЛ / без указания; офис, гибрид, удалёнка и их сочетания; 17 городов. Построчные SHA-256 всех 60 файлов — `backend/eval/briefs_test2.sha256` (проверка: `cd backend/eval/briefs && sha256sum -c ../briefs_test2.sha256`).

SHA-256 манифеста `backend/eval/briefs_test2.sha256` (хэш брифов test2): `fa766490b26e3c4e6f38900f3081d6456a15e45dfd25bbcf47fb6efbcbddb1c4`.

> **Поправка о переводе строки (9 октября 2026, после прогона; на план, брифы и числа не влияет).** Построчные хэши брифов и хэши `population.py` и `metrics.py` в §8.4 сняты в рабочей копии Windows, где файлы были с переводом строки CRLF; конфиги в §8.4 сняты с LF. Git хранит и выдаёт все текстовые файлы с LF (`.gitattributes: eol=lf`), поэтому `sha256sum -c` в свежей копии на Linux показывает «не совпадает» при неизменном содержимом. Проверка в коде (`test2_freeze_check`) сверяет по каноническому CRLF: из `backend/` — `uv run python -c "from eval.matching_eval import test2_freeze_check as c; print(c())"` (ожидается `ok: True`, 60 файлов, расхождений 0). Отдельный файл из §8.4 — `python3 -c "import hashlib,sys; b=open(sys.argv[1],'rb').read().replace(b'\r\n',b'\n'); print('LF ', hashlib.sha256(b).hexdigest()); print('CRLF', hashlib.sha256(b.replace(b'\n',b'\r\n')).hexdigest())" <путь>`: одна из двух строк совпадает с таблицей.

Единственная правка кода после заморозки — интеграция test2 в отчёт (§8.6): новые срезы `test2` и `test_all`, исключение test2 из прежних срезов `all` / `checkset`, чтобы ни одно прежнее число не изменилось. Логика метрик, оракул, популяция и конфиг не трогаются; соответствие проверяется сравнением прежних чисел до/после.

8.6. Результат (вписан после единственного прогона на test2; 9 октября 2026, 09:20 MSK)

Прогон: `uv run python -m eval.validate --workers 4` (сид 20261005), один раз; затем `eval.fairness`, `eval.market_sim`, `eval.checkset_eval` в этом порядке. Заморозка при прогоне подтверждена (`ME.test2_freeze_check()`: хэш манифеста совпал, 60 из 60 файлов совпали). Числа по b01–b40 (test, dev, all, checkset, устойчивость, разбор брифа) не изменились побитно — сравнены с `validation.json` до прогона.

Метрика (test2, 60 брифов)	Наш	B2	B0 (случайный)
P@10	0,540 [0,472; 0,615]	0,352 [0,277; 0,427]	0,167 [0,125; 0,212]
nDCG@10	0,554	0,377	0,203

- **Главная проверка (§8.3): разность P@10 с B2 = +0,188, 95% ДИ [0,128; 0,245], нижняя граница > 0 — гипотеза подтверждена.** Парный t-критерий: t = 6,32, p < 0,001. Разность nDCG@10 с B2 +0,177 [0,132; 0,222]; P@10 против B0 +0,373 [0,305; 0,442].
- Объединение test + test2 (80 брифов): P@10 0,557 против 0,380, разность +0,178 [0,121; 0,238].
- **Неудобное, но записанное:** абсолютный P@10 на test2 (0,54; нижняя граница ДИ 0,47) **ниже порога 0,60** из §2.3; на b01–b40 было 0,61–0,66. По специализациям (описательно): Backend 0,64 против 0,43 (+0,21), Frontend 0,55 против 0,31 (+0,24), **Fullstack 0,32 против 0,28 (+0,04, ДИ [−0,03; 0,13] — преимущество не доказано, 12 брифов)**.
- Эффект на новых данных (+0,188) оказался больше, чем в оценке для расчёта мощности (+0,145); оценка мощности была консервативной, не наоборот.
- Score, конфиг, оракул, популяция, брифы не менялись. Единственные правки кода после заморозки — интеграция test2 в отчёт (`eval/matching_eval.py`: срезы `test2` / `test_all`, `test2_freeze_check`, парный t-критерий, исключение test2 из прежних срезов `all` и `checkset`; `eval/report.py`: раздел 4.1a) и тесты (`tests/test_eval_matching.py`); хэши логики метрик (`metrics.py`), оракула, популяции и конфигов остались прежними. Хэш `matching_eval.py`, указанный в §8.4, после этих правок изменился — это известное и записанное расхождение.